Library Seat Reservation System

Public Technical Report

An anonymized portfolio edition of a data structures coursework prototype

This report explains the implemented architecture, the reasoning behind its core data structures, the verification performed on the public code, and the limits of the prototype.

Executive Summary

The project is a Java command-line application for managing library seat reservations. Each seat owns an AVL tree of non-overlapping time intervals, while hash maps provide direct access to seats and students. A stack records temporary seat-state transitions, and a weighted heuristic ranks available seats against a student's preferences.

The public edition corrects several issues found in the coursework submission: sample accounts now use fictional identifiers, on-site occupation works correctly, releasing a temporarily-away seat restores the available state, invalid time ranges are rejected, recommendation claims match the implemented weighted scoring algorithm, and the benchmark no longer claims to store 1,000 reservations within a 24-hour whole-hour model.

This is an educational prototype rather than a production reservation service. It runs in one process, stores data only in memory, and does not implement authentication, persistence, networking, or concurrency control.

Objectives and Scope

- Demonstrate a self-balancing binary search tree in a concrete reservation problem.
- Prevent overlapping bookings for the same seat and time interval.
- Recommend available seats using explicit, explainable preference weights.
- Model occupied and temporarily-away states with last-in, first-out behavior.
- Provide a dependency-free Java program that can be compiled with the JDK alone.

System Architecture

The application separates terminal interaction, coordination logic, entities, and data structures. The design remains intentionally small so that the behavior of each structure is visible to a student or reviewer.

Component	Responsibility	Primary structure
LibrarySystem	Reads commands, validates time ranges, and presents results	Scanner and menu loop
SeatManager	Coordinates seats, students, reservations, states, and recommendations	HashMap
Seat	Owns one seat's schedule, attributes, and live occupancy state	AVLTree and Stack
AVLTree	Stores non-overlapping TimeSlot objects and checks conflicts	Self-balancing BST
Student	Stores a fictional account and its seat preferences	HashSet
PerformanceTest	Runs small illustrative microbenchmarks	JDK timing APIs

```
LibrarySystem
    -> SeatManager
        -> HashMap<String, Student>
        -> HashMap<String, Seat>
            -> AVLTree<TimeSlot>
            -> Stack<SeatStatus>
```

Reservation Index

Time Interval Model

A TimeSlot stores a start hour, end hour, and student identifier. Valid values satisfy 0 <= start < end <= 24. Two intervals overlap unless one ends at or before the other begins. This means 09:00-10:00 and 10:00-11:00 are adjacent but not conflicting.

```
overlap(a, b) = not (a.end <= b.start or a.start >= b.end)
valid(start, end) = 0 <= start < end <= 24
```

AVL Tree Operations

Time slots are ordered by start time and then by end time. Before insertion, the tree checks for an overlap. Because the stored intervals never overlap, the search can choose the left or right subtree from the interval boundaries. Standard AVL rotations restore balance after insertions and deletions.

Operation	Purpose	Expected complexity
Conflict check	Find whether a proposed interval overlaps an existing reservation	O(log n)
Insert	Add a non-conflicting interval and rebalance the tree	O(log n)
Delete	Remove a matching interval and rebalance the tree	O(log n) typical
List all	Produce reservations in chronological order	O(n)

The logarithmic conflict-check statement depends on the non-overlapping-interval invariant. The public documentation states that condition explicitly instead of presenting the complexity as unconditional.

Recommendation and Seat State

Preference-Weighted Recommendation

The recommendation function first excludes seats with a conflicting reservation. It then adds configured weights for attributes that match the student's preferences, applies a small penalty to an unrequested near-door seat, and adds a small bonus to a seat with fewer than three reservations. Candidates are sorted by descending score and then by seat ID for deterministic ties.

Attribute	Weight when requested	Interpretation
Power outlet	+3	Highest-weight convenience feature

Attribute	Weight when requested	Interpretation
Window	+2	Natural-light preference
Quiet area	+2	Noise preference
Near restroom	+1	Location convenience
Near door	-1 if not requested	Small penalty for possible traffic
Low reservation count	+1	Tie-shaping availability bonus

This is a transparent weighted heuristic. It is not dynamic programming, machine learning, or an adaptive recommender. The public README and program output use the same terminology as the implementation.

Stack-Based Occupancy State

Every seat begins with an available state. Occupation pushes an occupied state, temporary leave pushes a temporarily-away state, and return pops that temporary state. Releasing a seat now clears any temporary-away layer and restores the base available state. The manager checks ownership before release.

```
available
available -> occupied(student)
available -> occupied(student) -> temporarily_away(student)
release -> available
```

Verification

The public edition was compiled with JDK 23 and exercised with a dependency-free smoke test. The test uses assertions and covers reservation conflicts, cancellation, recommendation order, occupation, temporary leave, return, release, invalid durations, invalid recommendation counts, and invalid time ranges.

Verification case	Expected behavior	Result
Compile all source	No compiler errors	Passed
Reject overlapping reservation	Second booking returns false	Passed
Rank matching seat first	Preference match receives the top position	Passed
Occupy available seat	Status changes to occupied	Passed
Temporary leave and return	Stack restores occupied state	Passed
Release while away	Seat returns to available	Passed
Reject invalid input	Invalid time or count is not accepted	Passed

Illustrative Benchmarks

The repository includes microbenchmarks for AVL conflict checks, recommendation over up to 200 seats, stack operations, and an AVL-versus-linear-list comparison. The revised benchmark uses the same non-overlapping intervals for both comparison structures and reports the number of intervals actually stored.

Timing values are intentionally not treated as stable project results. They vary with the computer, Java runtime, just-in-time compilation, warm-up, and background load. The benchmark is useful for experimentation, but a production performance study would require a dedicated framework such as JMH, repeated forks, warm-up iterations, controlled data generation, and statistical reporting.

Limitations and Future Work

Current limitation	Recommended next step
All data is in memory	Add a database and transactional repository layer
Single-process command-line interface	Add a service API and browser or mobile client
No authentication or authorization	Introduce secure identity and role-based access
No concurrent booking control	Use atomic transactions and conflict-safe writes
Whole-hour time resolution	Use date-time intervals with time-zone handling
Heuristic recommendation	Validate weights with users or add a configurable policy
Small assertion-based test suite	Add unit, property-based, and integration tests

Reproducibility

Compile and run the application from the repository root:

```
mkdir -p build/classes
javac -d build/classes src/*.java
java -cp build/classes LibrarySystem
```

Compile and run the smoke test:

```
mkdir -p build/test-classes
javac -d build/test-classes src/*.java tests/SystemSmokeTest.java
java -ea -cp build/test-classes SystemSmokeTest
```

Conclusion

The project demonstrates how multiple data structures can cooperate in a compact application: AVL trees index reservation intervals, hash maps locate entities, a stack models reversible occupancy states, and a deterministic weighted heuristic ranks seats. The public edition improves correctness, privacy, reproducibility, and the accuracy of its technical claims while preserving the educational purpose of the original work. It omits personal identifiers, instructor information, institution branding, local IDE metadata, submission instructions, and screenshots containing private data; all sample users are fictional.